

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – PSEUDOCODE WRITING TASK

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 1 – Pseudocode Writing Task
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	<i>Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.</i>		
Student Name:	Shahana	Reg. No.:	192524479
Section / Batch:	Slot A	Date:	31/08/2026

Code of Conduct

I Shahana (192524479) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Shahana

Instructions

- This test contains 5 Pseudocode Writing Task questions. Total: 100 Marks.
- When a student answer using pseudocode writing task should evaluate based on the ability to design clear, logical, and efficient pseudocode solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. KRUSKAL'S ALGORITHM:

Q1 Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Write pseudocode for constructing an MST using Kruskal's Algorithm. A city planning department needs to connect multiple zones with underground fiber-optic cables. Each zone is represented as a vertex, and possible cable connections between zones are represented as edges with installation costs (weights).

Constraints:

- All zones must be connected
- Total installation cost must be minimized
- No redundant connections (no cycles) allowed
- The network must remain fully connected

Your Answer

Step 1: Initialize

- Create an empty list MST_Edges to store the final network
- Sort all edges E in non-decreasing order of their installation cost (weight)
- Create a Disjoint Set (Union-Find) structure:
For each zone (vertex) v in V :
 MAKE-SET(v) // each zone starts as its own separate component

Step 2: Process edges in increasing order of cost

- FOR each edge (u, v) with weight w in sorted E :
 IF FIND-SET(u) $\neq$ FIND-SET(v):
 // Adding this edge does not form a cycle
 ADD edge (u, v) to MST_Edges
 UNION(u, v) // merge the two zones into one connected component
 ELSE:
 // u and v are already connected, adding this edge creates a cycle
 DISCARD edge (u, v)

Step 3: Stop condition

- REPEAT Step 4 until MST_Edges contains $(V - 1)$ edges
(i.e., all zones are connected with minimum possible edges)

2. TOPOLOGICAL SORT:

Q2 Graph - Topological Sort

A DAG represents task dependencies. Write pseudocode for Topological Sort using DFS with cycle detection.

Constraints:

- Detect cycle before sorting
- Use DFS approach

Your Answer

ALGORITHM TopologicalSort(Graph G)

visited[] tracks nodes: 0 = unvisited, 1 = in progress, 2 = completed

stack[] stores the final topological order

hasCycle flag indicates if a cycle is detected

INITIALIZE visited[1...n] = 0 for all nodes

INITIALIZE stack = empty list

INITIALIZE hasCycle = false

FOR each node v in G :

 IF visited[v] == 0 THEN

 DFS_Visit(v)

```

    END IF
  END FOR

  IF hasCycle == true THEN
    PRINT "Cycle detected, topological sort not possible"
    RETURN
  ELSE
  END FOR

  visited[u] = 2    // mark as fully processed
  PUSH u onto stack // add to result after all descendants are done

END FUNCTION

```

3. SHORTEST PATH ALGORITHM:

Q3 Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's

Write pseudocode integrating MST + Dijkstra for Smart transportation system.

Constraints:

- Use MST for infrastructure
- Use shortest path for routing

Your Answer

- Create an empty set MST_Edges
- Sort all edges of G in increasing order of weight (cost)
- Initialize Disjoint Set (Union-Find) for all stations
 - for each station S in G:
 - MAKE-SET(S)
- FOR each edge (u, v) with weight w in sorted edge list:

```

    IF FIND-SET(u) != FIND-SET(v) THEN
      ADD edge (u, v) to MST_Edges
      UNION(u, v)
    END IF
  END FOR

```

- RETURN MST_Edges // This forms the minimum-cost road/rail network connecting all stations

END

ALGORITHM Find_Shortest_Route(MST_Graph, source)

INPUT: MST_Graph (built from Part A), source station

- Create distance array dist[] and set all values to INFINITY
dist[source] = 0

- Create a priority queue PQ and insert (source, 0)

- WHILE PQ is not empty:

```

  FOR each neighbor v of u in MST_Graph:
    newDist = dist[u] + weight(u, v)
    IF newDist < dist[v] THEN
      dist[v] = newDist
      INSERT/UPDATE (v, newDist) in PQ
    END IF
  END FOR

```

4. DIJKSTRA'S ALGORITHM:

Q4 Shortest Path Algorithm - Dijkstra's Algorithm

Shortest Path with "Toll-Free" Voucher: You have a voucher that allows you to traverse one edge for free (weight = 0). Write pseudocode to find the shortest path from S to D utilizing this voucher optimally.

Your Answer

```
BEGIN
  for each vertex v in G:
    dist[v][0] = INFINITY
    dist[v][1] = INFINITY

  dist[S][0] = 0

  PQ = empty priority queue
  PQ.insert((0, S, 0))

  while PQ is not empty:
    (d, u, used) = PQ.extractMin()

    if d > dist[u][used]:
      continue // skip outdated entry

    for each edge (u, v) with weight w in G.adjacency[u]:

      // Option 1: Do not use voucher on this edge
      if dist[u][used] + w < dist[v][used]:
        dist[v][used] = dist[u][used] + w

    // Option 2: Use voucher on this edge (if not already used)
    if !used && (dist[u][0] + 0 < dist[v][0]):
      dist[v][0] = dist[u][0]

  // Final answer is the minimum of both states at destination
  answer = MIN(dist[D][0], dist[D][1])

  return answer
END
```

5. MINIMUM SPANNING TREE:

ALGORITHM SecondBestMST(Graph G)

Step 1: Find the initial Minimum Spanning Tree (MST)

MST_Edges = Kruskal(G)

MST_Weight = CalculateWeight(MST_Edges)

second_best_weight = INFINITY

Step 2: Iterate through each edge in the MST

FOR each edge e IN MST_Edges DO

// Create a temporary graph excluding the current edge from the MST

Temp_Graph = G.copy()

Temp_Graph.removeEdge(e)

Step 3: Find the MST of the graph without edge 'e'

Note: To find the "next best", we find a new MST that replaces 'e' with the best available alternative edge.

New_MST = Kruskal(Temp_Graph)

Check if a valid MST was still formed

IF New_MST.edges.count == G.vertices.count - 1 THEN

Step 4: Update the second best weight

IF Current_Weight < second_best_weight THEN

second_best_weight = Current_Weight

END IF

END IF

END FOR